Matrix Factorization: SVD and ALS

Discovering hidden factors behind preferences

Session 3 | 13:30–14:15 | Theory

ACM Student Chapter Workshop

Where similarity-based CF hurt (Lab 2 recap)

What you saw in Lab 2	Why it hurts	What we wish for
Neighbours share only 2–19 movies	similarity from tiny overlap	dense representation use *all* the data
$k = 20$ gave the same list as $k = 50$	sparsity caps the neighbourhood	
610×610 and $1\,921 \times 1\,921$ matrices	does not scale to millions	few numbers per entity
Coverage only 75–95% of test pairs	some cells get no answer	every cell predicted
User/item effects fixed by centring	ad hoc	model biases *explicitly*

Question

CF compares users or items pair by pair. Can we instead discover hidden factors that explain preferences?

Can a much smaller object explain the whole matrix?

$$R = \begin{bmatrix} 5 & 4 & ? & 1 \\ 4 & ? & 5 & ? \\ ? & 2 & 4 & 5 \end{bmatrix}$$

Only 8 of the 12 cells are known.

- Users are not 4 independent numbers: *taste* is driven by a few underlying factors
- If each user and each item can be described by $k = 2$ numbers, we need $3 \times 2 + 4 \times 2 = 14$ numbers...
- ... and then every cell has a prediction

Low-rank assumption

R is approximately a product of a tall thin matrix and a short wide one:

$$R \approx P\,Q^{\top}$$

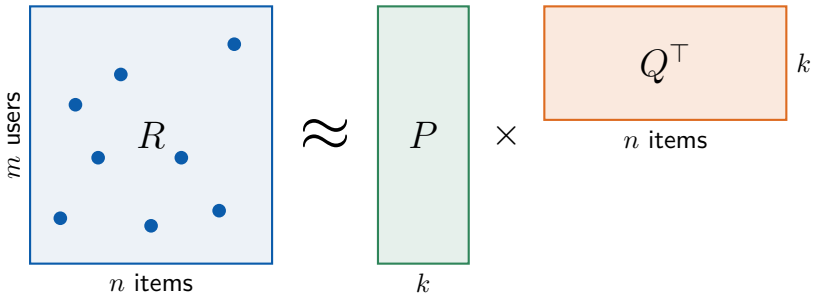
P: user factors $\quad Q$: item factors

Parameter count

$m \cdot n$ cells $\longrightarrow$ $(m+n)\,k$ numbers

MovieLens-small, $k = 50$: 5.9 M cells vs $\approx$ 0.52 M parameters

Matrix factorization: $R \approx PQ^\top$



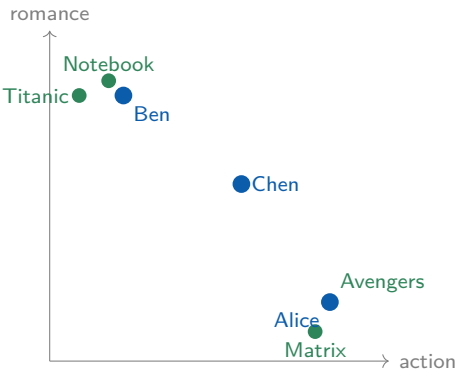
each row of P: a user
$p_u \in \mathbb{R}^k$

each row of Q: an item
$q_i \in \mathbb{R}^k$

Reading it

Instead of saying "user 1 is similar to user 2", we describe *every user and item by k hidden numbers*. The prediction for cell (u, i) is then $\hat{r}_{ui} = \boldsymbol{p}_u^\top \boldsymbol{q}_i$.

Latent factors: an intuition (and a warning)



Movies (green) and users (blue, vectors drawn $\div 5$) in a 2-D latent space.

- Movie q_i: how *action-ish*, how *romance-ish*
- User p_u: how much they *like* action, romance
- Close together $\Rightarrow$ high predicted rating

Warning

Real factors are not guaranteed to be genres. They are whatever directions best explain the ratings. Treat them as **latent dimensions**, not labels.

Prediction = user vector · item vector

$$\hat{r}_{ui} = \boldsymbol{p}_u^\top \boldsymbol{q}_i = \sum_{f=1}^{k} p_{uf}\, q_{if}$$

P (users $\times$ 2): Alice $(4.5, 1.0)$, Ben $(1.0, 4.5)$, Chen $(3, 3)$
Q (movies $\times$ 2): Matrix $(.9, .1)$, Titanic $(.1, .9)$, Notebook $(.2, .95)$, Avengers $(.95, .2)$

$\hat{R} = PQ^\top$	Matrix	Titanic	Notebook	Avengers
Alice	**4.15**	1.35	1.85	**4.48**
Ben	1.35	**4.15**	**4.48**	1.85
Chen	3.00	3.00	3.45	3.45

By hand: Alice $\times$ Notebook

$4.5{\cdot}0.2 + 1.0{\cdot}0.95 = 0.90 + 0.95 = 1.85$

Geometry

$p^\top q = \|p\|\|q\| \cos\theta$: large when the vectors point the same way and have large length

Adding biases: the model you will actually train

$$\hat{r}_{ui} = \underbrace{\mu}_{\text{global mean}} + \underbrace{b_u}_{\text{user habit}} + \underbrace{b_i}_{\text{item quality}} + \underbrace{\boldsymbol{p}_u^\top \boldsymbol{q}_i}_{\text{taste match}}$$

- Lab 1: users differ in how they rate; movies differ in quality
- Without b_u, b_i the dot product must *waste factors* on average effects
- The bias-only model $\mu + b_u + b_i$ is already a strong baseline

Example (Lab 3 output)

User 10, *Toy Story*:
$\mu = 3.50$
$b_u = -0.45$ (harsh rater)
$b_i = +0.39$ (well-liked movie)
$p_u^\top q_i = +0.08$
$\hat{r} = 3.50 - 0.45 + 0.39 + 0.08 = \mathbf{3.52}$

Why should low rank work? (optional)

- People's tastes have structure: a few themes (genre, era, mood, "mainstream-ness") explain a lot
- Many users $\times$ many items provide *redundant* information about those themes
- Fitting $k \ll \min(m, n)$ numbers per entity *forces* the model to share statistical strength
- Neighbourhood CF uses pairwise evidence; MF pools evidence across *all* users and items

Netflix Prize (2006–09)

Matrix-factorization models were the strongest single models and the backbone of the winning ensembles (Koren, Bell, Volinsky 2009).

Trade-off

k too small: underfit (cannot represent taste)
k too large: overfit (memorises noise)

Why is it called “SVD”?

Linear-algebra SVD (complete matrix):

$$A = U \Sigma V^\top$$

Eckart–Young: keep the top k singular values $\Rightarrow$ the best rank-k approximation of A (in least squares).

Problem for ratings:

- R has missing entries; SVD needs a complete matrix
- filling “?” with 0 or the mean biases the result

What recommender “SVD” means

Fit P and Q *only on observed ratings*:

$$\min_{P,Q} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^\top q_i)^2 + \text{reg.}$$

Popularised by Simon Funk (Netflix Prize blog, 2006), hence “FunkSVD”. Surprise’s `SVD` is the biased version of this.

No singular-value decomposition is computed: only the *idea* of low-rank factors is inherited.

Learning the factors: the objective

$$\min_{P,Q,b} \underbrace{\sum_{(u,i)\in\Omega} \left(r_{ui} - \mu - b_u - b_i - \boldsymbol{p}_u^\top \boldsymbol{q}_i\right)^2}_{\text{reconstruction error on known ratings}} + \underbrace{\lambda\Big(\|\boldsymbol{p}_u\|^2 + \|\boldsymbol{q}_i\|^2 + b_u^2 + b_i^2\Big)}_{\text{regularisation}}$$

1. Fit

Make predictions match the *observed* ratings Ω (not the empty cells)

2. Regularise

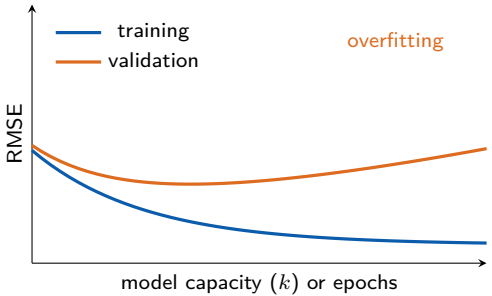
Penalise large vectors/biases; λ controls how hard (`reg_all`)

3. Capacity k

Number of factors (`n_factors`); fixed before training

Ω: set of observed (u, i) pairs. The problem is non-convex in (P, Q) jointly, but convex in P for fixed Q and vice versa.

Overfitting and the role of regularisation



Schematic. In Lab 3 you will see that more factors do *not* always help.

- **Overfitting:** training error keeps dropping, error on unseen ratings rises
- More factors or epochs $\Rightarrow$ more capacity $\Rightarrow$ easier to memorise noise
- λ larger $\Rightarrow$ smaller parameters $\Rightarrow$ smoother model

Practical recipe

Tune k, λ, γ, epochs by cross-validation on the training set; evaluate once on the test set.

Optimisation 1: gradient and the SGD update

Let $e_{ui} = r_{ui} - \hat{r}_{ui}$ and take the loss on one rating (absorbing factor 2 into γ, λ):

$$\ell_{ui} = \tfrac{1}{2} e_{ui}^2 + \tfrac{\lambda}{2}(\|\boldsymbol{p}_u\|^2 + \|\boldsymbol{q}_i\|^2 + b_u^2 + b_i^2)$$

$$\frac{\partial \ell_{ui}}{\partial \boldsymbol{p}_u} = -e_{ui}\,\boldsymbol{q}_i + \lambda\,\boldsymbol{p}_u \qquad \frac{\partial \ell_{ui}}{\partial \boldsymbol{q}_i} = -e_{ui}\,\boldsymbol{p}_u + \lambda\,\boldsymbol{q}_i \qquad \frac{\partial \ell_{ui}}{\partial b_u} = -e_{ui} + \lambda b_u$$

Move against the gradient with learning rate γ:

SGD update for one observed rating (u, i)

$$b_u \leftarrow b_u + \gamma\,(e_{ui} - \lambda b_u) \qquad \boldsymbol{p}_u \leftarrow \boldsymbol{p}_u + \gamma\,(e_{ui}\,\boldsymbol{q}_i - \lambda\,\boldsymbol{p}_u)$$
$$b_i \leftarrow b_i + \gamma\,(e_{ui} - \lambda b_i) \qquad \boldsymbol{q}_i \leftarrow \boldsymbol{q}_i + \gamma\,(e_{ui}\,\boldsymbol{p}_u - \lambda\,\boldsymbol{q}_i)$$

Intuition: if the prediction is too low ($e > 0$) move p_u toward q_i (and q_i toward p_u); the $-\lambda\cdot$ term pulls everything gently back to zero.

SGD training loop

Algorithm

initialise $p_u, q_i \sim \mathcal{N}(0, 0.1^2)$, $b_u = b_i = 0$
repeat for `n_epochs`:
  shuffle the observed ratings
  for each (u, i, r_{ui}):
    $\hat{r} \leftarrow \mu + b_u + b_i + p_u^\top q_i$
    $e \leftarrow r_{ui} - \hat{r}$
    update b_u, b_i, p_u, q_i (previous slide)
return μ, b, P, Q

- Cost per epoch: $O(|\Omega|\, k)$ – linear in the number of ratings
- Needs: learning rate γ, epochs, λ
- Sequential and noisy, but cheap and easy to implement
- Result depends on the random initialisation $\Rightarrow$ report mean $\pm$ std over seeds

Worked SGD step (one rating)

Given $r_{ui} = 4$, $p_u = (0.50, 0.10)$, $q_i = (0.40, 0.30)$, $\gamma = 0.01$, $\lambda = 0.1$ (no biases, for clarity)

1. Prediction: $\hat{r} = 0.5 \cdot 0.4 + 0.1 \cdot 0.3 = 0.23$
2. Error: $e = 4 - 0.23 = 3.77$
3. $p_u \leftarrow p_u + \gamma(e\, q_i - \lambda p_u)$
   $= (0.5 + 0.01(3.77 \cdot 0.4 - 0.05),\ 0.1 + 0.01(3.77 \cdot 0.3 - 0.01))$
   $= (0.5146,\ 0.1112)$
4. $q_i \leftarrow q_i + \gamma(e\, p_u - \lambda q_i)$
   $= (0.4 + 0.01(3.77 \cdot 0.5 - 0.04),\ 0.3 + 0.01(3.77 \cdot 0.1 - 0.03))$
   $= (0.4185,\ 0.3035)$

New prediction

$\hat{r} = 0.5146 \cdot 0.4185 + 0.1112 \cdot 0.3035 \approx$ **0.249**

- The prediction moved from 0.23 toward 4; the error shrank from 3.77 to 3.75
- Each step is small (γ is small); it takes many passes over all ratings to converge

Check

Why did q_i move toward p_u's direction and not away?

Optimisation 2: Alternating Least Squares (ALS)

Idea. The objective is not convex in (P, Q) jointly, but for *fixed* Q it is a ridge regression in each p_u (and vice versa).

$$\min_{p_u} \sum_{i \in \Omega_u} (r_{ui} - p_u^\top q_i)^2 + \lambda \|p_u\|^2 \quad \Longrightarrow \quad \boxed{p_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top r_u}$$

Q_u: rows q_i for the items user u rated; r_u: those ratings. Symmetric formula for q_i.

Worked example ($k = 2$, $\lambda = 1$)

items rated: $q_1 = (1, 0)$, $q_2 = (0, 1)$, $q_3 = (1, 1)$; ratings $r = (4, 2, 5)$

$$Q_u^\top Q_u + \lambda I = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}, \quad Q_u^\top r = \begin{bmatrix} 9 \\ 7 \end{bmatrix}$$

$$p_u = \begin{bmatrix} 2.5 \\ 1.5 \end{bmatrix} \quad \text{(check: } \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 2.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 9 \\ 7 \end{bmatrix} \text{)}$$

ALS loop

init Q randomly
repeat
  fix Q: solve every p_u (closed form)
  fix P: solve every q_i (closed form)
until the objective stops decreasing

SVD (SGD) vs ALS

	SGD (FunkSVD / Surprise `SVD`)	ALS
Update	small gradient step per rating	exact solve of a ridge regression per user / item
Hyper-parameters	k, λ, γ, epochs	k, λ, iterations (no learning rate)
Cost per sweep	$O(\|\Omega\|k)$	$O(\|\Omega\|k^2 + (m+n)k^3)$
Parallelism	sequential (hogwild variants exist)	embarrassingly parallel over users, then items
Objective	noisy decrease	never increases each half-step
Strengths	simple, fast per step, flexible losses	stable; great for *implicit* feedback and clusters (Spark MLlib)

In one line

SVD (SGD): nudge the factors rating by rating. **ALS:** solve one side exactly while the other is frozen.

Evaluating predictions: MAE and RMSE

$$\text{MAE} = \frac{1}{N}\sum_{(u,i)} |r_{ui} - \hat{r}_{ui}| \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{(u,i)} (r_{ui} - \hat{r}_{ui})^2}$$

r	$\hat{r}$	$\lvert e\rvert$	e^2
4	3.8	0.2	0.04
3	3.4	0.4	0.16
5	4.1	0.9	0.81
2	2.5	0.5	0.25
4	4.4	0.4	0.16

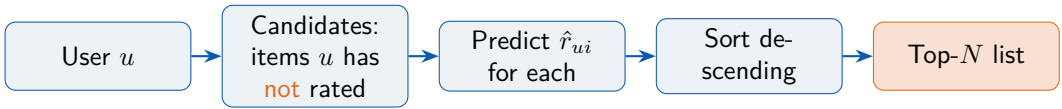
MAE $= 2.4/5 =$ **0.48**
RMSE $= \sqrt{1.42/5} =$ **0.533**

RMSE punishes large errors more

Errors $(0, 0, 0, 0, 2)$: MAE $= 0.40$, RMSE $= 0.89$.

- Lower is better, in stars
- MAE: "typical error"; RMSE: outlier-sensitive
- Always compare with a baseline

From predicted ratings to a Top-N list



Example (user 10)

1. Movie A 4.72
2. Movie B 4.61
3. Movie C 4.55
4. Movie D 4.48
5. Movie E 4.42

Ranking metrics

$$\mathrm{P@}N = \frac{|\mathrm{rel} \cap \mathrm{top}\text{-}N|}{N}$$

$$\mathrm{R@}N = \frac{|\mathrm{rel} \cap \mathrm{top}\text{-}N|}{|\mathrm{rel}|}$$

rel = items with test rating ≥ 4

Rating prediction $\neq$ **Top-N**: a low RMSE does not guarantee a good top of the list.

Implicit feedback: how MF extends (optional)

Problem: no ratings, only observed behaviour; unobserved $\neq$ negative.

Hu–Koren–Volinsky (2008):

- preference $x_{ui} = 1$ if u interacted with i, else 0
- confidence $c_{ui} = 1 + \alpha\, n_{ui}$ (more interactions $\Rightarrow$ more trust)
- minimise $\sum_{\text{all } (u,i)} c_{ui}\,(x_{ui} - p_u^\top q_i)^2 + \lambda(\cdots)$
- solved by ALS (that is why ALS is popular)

Other approaches (Day 2 / further reading)

Bayesian Personalized Ranking (pairwise loss), neural collaborative filtering, sequence-aware models, two-tower retrieval

Cautions: interpretation and uniqueness

Rotation ambiguity

For any invertible $A \in \mathbb{R}^{k \times k}$,

$$PQ^\top = (PA)(QA^{-\top})^\top$$

so the *same* predictions arise from different factors $\Rightarrow$ individual axes have no fixed meaning; only geometry (who is near whom) does.

- Non-convex: different seeds, different factors, similar RMSE
- Cold start remains: a new user/item has no vector $\Rightarrow$ fall back to $\mu + \ldots$ or content
- Popularity bias remains; the model learns what the data shows
- Explanations: "because users like you…" is harder than in neighbourhood CF

From maths to Surprise: parameter dictionary

Parameter	Meaning	Symbol	Default
`n_factors`	latent dimension	k	100
`n_epochs`	SGD passes over Ω	–	20
`lr_all`	learning rate	γ	0.005
`reg_all`	L2 regularisation	λ	0.02
`biased`	include μ, b_u, b_i	–	True
`random_state`	seed for init	–	None

- `model.pu` $= P$ (users $\times k$), `model.qi` $= Q$ (items $\times k$), `model.bu`, `model.bi` $= b_u, b_i$
- `model.predict(uid, iid).est` $= \hat{r}_{ui}$ (clipped to the `Reader` rating scale)
- Per-group variants: `lr_bu`, `lr_bi`, `lr_pu`, `lr_qi`, `reg_bu`, ...

Prepare for Lab 3: build the SVD recommender (14:15–15:45)

1. **Load** the split into Surprise (`Reader`)
2. **Baselines**: random, $\mu + b_u + b_i$
3. **Train SVD** (`n_factors=50`)
4. **Predict**: actual vs predicted table
5. **Evaluate**: RMSE, MAE
6. **Open the model**: P, Q, biases
7. **Top-N** for user 10
8. **Experiment**: `n_factors`, tuning
9. **Precision@10** vs popularity

Predict before you run

1. Will SVD beat the $\mu + b_u + b_i$ baseline by more than 0.05 RMSE?
2. Does 200 factors beat 20?
3. Who wins Precision@10: SVD or popularity?

Pitfalls

rating scale (0.5–5), not (1–5)
raw vs inner ids
tune on train (CV), not test

Summary and check questions

- $R \approx PQ^\top$: describe every user and item by k **latent factors**
- Prediction: $\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$ – fills every cell
- Learn by minimising squared error on observed ratings $+\lambda \cdot$ size
- **SGD** (FunkSVD): small steps per rating; **ALS**: alternate exact ridge solves
- Evaluate with RMSE / MAE *and* Top-N metrics, always against baselines
- Watch for overfitting: tune k, λ, γ with cross-validation

Check yourself

1. What does $R \approx PQ^\top$ mean?
2. Why add b_u and b_i?
3. What does λ do?
4. SGD vs ALS in one sentence each?
5. Why can RMSE and Precision@10 disagree?

References and further reading I

- Y. Koren, R. Bell, C. Volinsky, Matrix factorization techniques for recommender systems, *IEEE Computer* 42(8), 2009.
- S. Funk, Netflix update: try this at home (FunkSVD), blog post, 2006.
- Y. Koren, Factorization meets the neighborhood, *KDD*, 2008.
- Y. Hu, Y. Koren, C. Volinsky, Collaborative filtering for implicit feedback datasets, *ICDM*, 2008.
- Y. Zhou, D. Wilkinson, R. Schreiber, R. Pan, Large-scale parallel collaborative filtering for the Netflix Prize, *AAIM*, 2008.
- S. Rendle et al., BPR: Bayesian personalized ranking from implicit feedback, *UAI*, 2009.
- C. Eckart, G. Young, The approximation of one matrix by another of lower rank, *Psychometrika*, 1936.
- N. Hug, Surprise: a Python library for recommender systems, *JOSS* 5(52), 2020; documentation: surpriselib.com.
- P. Cremonesi, Y. Koren, R. Turrin, Performance of recommender algorithms on top-N recommendation tasks, *RecSys*, 2010.